同等学力申硕计算机综合软件工程 2020-2025 去重合并复习稿

来源文档：

- `SoftwareEngineering/2020.md`
- `SoftwareEngineering/2021.md`
- `SoftwareEngineering/2022.md`
- `SoftwareEngineering/2023.md`
- `SoftwareEngineering/2025.md`

整理原则：

1. 重复出现的内容合并为一个高频考点。
2. 同类题型归到同一模块，按解题方法组织，不按年份组织。
3. 每个模块包含：最终需要学习的知识点、解题技巧、练习题、参考答案。
4. 高频模块优先：生命周期、需求与设计、内聚耦合、测试、DFD、状态图、CMM/CMMI、面向对象。

一、最终必学知识点地图

优先级	模块	必学内容
★★★★★	生命周期与需求设计	需求分析、概要设计、详细设计、编码、测试、维护的任务和产物
★★★★★	软件测试	黑盒/白盒、测试阶段、覆盖准则、测试目的、McCabe 复杂度、基本路径覆盖
★★★★★	结构化分析与 DFD	DFD 四元素、顶层/0 层/1 层 DFD、数据平衡、数据字典、建模步骤
★★★★★	面向对象	类与对象、封装、继承、多态、消息传递、动态绑定、UML 关系、状态图、顺序图

优先级	模块	必学内容
★★★★☆	软件设计	高内聚低耦合、内聚类型、耦合类型、结构化程序三结构、N-S 图、PAD 图
★★★★☆	软件过程模型	瀑布、增量、演化、螺旋、喷泉、敏捷、RUP、RAD
★★★★☆	过程改进与管理	CMM、CMMI、配置管理、质量控制、风险管理
★★★☆☆	补充考点	开源软件、Web 应用软件、设计模式、编程风格、软件维护类型

二、考前速查表

2.1 阶段任务速查

阶段	核心问题	主要任务	常见产物
可行性研究	值不值得做	技术、经济、操作、法律可行性分析	可行性研究报告
需求分析	软件要做什么	获取、分析、定义需求，建立逻辑模型	SRS、用例图、DFD、ER 图
概要设计	系统总体怎么分	划分模块、确定体系结构、模块接口	软件结构图、模块结构图
详细设计	模块内部怎么实现	设计算法、数据结构、控制流程	详细设计说明书、N-S 图、PAD 图
编码	如何写成程序	用语言实现设计结果	源代码
测试	软件是否正确	发现错误、验证需求是否满足	测试报告、缺陷报告
维护	交付后如何修改	纠错、适应、完善、预防	新版本、维护记录

高频判断：

需求分析解决“做什么”，不是“怎么做”。
概要设计确定总体结构和模块关系。
详细设计确定模块算法和内部流程。
测试计划通常在需求分析阶段制定。

2.2 测试速查

概念	关键词
黑盒测试	不看内部结构，按需求测功能
白盒测试	看内部逻辑，测语句、分支、条件、路径
单元测试	测单个模块内部逻辑
集成测试	测模块接口、调用、协作
系统测试	测完整系统是否满足需求
验收测试	用户/客户确认能否交付
语句覆盖	每条语句至少执行一次
判定覆盖	每个判断真假分支都至少一次
条件覆盖	每个条件真假都至少一次
路径覆盖	所有可能路径至少一次
McCabe	`V(G) = 判定节点数 + 1 = E − N + 2`

2.3 DFD 速查

元素	含义	常见表示
外部实体	系统外部的人、组织、其他系统	矩形
加工	对数据进行处理的功能	圆形/圆角矩形
数据流	数据的流动	带名称箭头
数据存储	文件、数据库、数据表	双横线/开口矩形

高频判断：

```
顶层 DFD 又叫系统环境图，只画一个系统加工。
0 层 DFD 是对顶层总加工的分解。
父图和子图必须保持数据平衡。
外部实体不能直接连数据存储，必须经过加工。
外部实体之间一般不能直接连线。
```

2.4 面向对象速查

概念	一句话
类	对一组对象共同属性和行为的抽象
对象	类的具体实例
封装	数据和操作绑定，隐藏内部实现
继承	子类复用父类属性和方法
多态	同一消息，不同对象，不同响应
重载	同类中方法名相同，参数列表不同
重写	子类重新定义父类已有方法
消息传递	对象之间的交互机制
动态绑定	运行时决定调用哪个具体方法

2.5 内聚与耦合速查

内聚从低到高：

```
偶然 < 逻辑 < 时间 < 过程 < 通信 < 顺序 < 功能
```

耦合从低到高常见理解：

```
非直接耦合 < 数据耦合 < 标记耦合 < 控制耦合 < 外部耦合 < 公共耦合 < 内容耦合
```

答题原则：

好的模块设计追求高内聚、低耦合。

三、生命周期、需求与设计

3.1 软件生命周期阶段

必须掌握

生命周期题最常见的考法是给出一项活动，让你判断属于哪个阶段。

题干关键词	对应阶段
用户需求、功能需求、非功能需求、逻辑模型、SRS	需求分析
总体结构、模块划分、模块接口、体系结构	概要设计
模块算法、数据结构、处理流程、N-S 图、PAD 图	详细设计
源程序、程序语言实现	编码
发现错误、验证需求、执行测试用例	测试
修改已交付软件、适应环境、增强功能、修复缺陷	维护

解题技巧

1. 看到“软件应当做什么”，选需求分析。
2. 看到“系统分成哪些模块”，选概要设计。
3. 看到“模块内部算法如何写”，选详细设计。
4. 看到“交付后修改”，选维护。
5. 看到“测试计划”，优先选需求分析阶段制定。

练习题与答案

题 1： “确定系统模块划分、模块之间接口和软件总体结构”属于哪个阶段?

答案：概要设计。

题 2： “定义模块内部算法和处理流程”属于哪个阶段？

答案：详细设计。

题 3： 判断：需求分析阶段主要给出软件的具体处理算法。

答案：错。处理算法属于详细设计，需求分析关注软件要做什么。

3.2 需求分析

必须掌握

需求分析的主要任务是：

> 理解用户需求，定义功能需求和非功能需求，建立系统逻辑模型，编写需求规格说明书。

需求类型：

类型	含义	例子
功能需求	系统必须提供的功能	登录、查询、支付、生成报表
非功能需求	性能、可靠性、安全性等质量要求	2 秒内响应、支持 1000 人并发
约束条件	开发或运行必须满足的限制	必须使用某数据库、必须兼容某浏览器

答题模板

> 需求分析阶段的主要任务是定义软件需求并建立系统逻辑模型，它关注软件“做什么”，主要产物是需求规格说明书和相

练习题与答案

题 1： 需求分析阶段的核心任务是什么？

答案：定义软件需求并建立系统逻辑模型。

题 2： “系统必须支持 HTTPS 访问，平均响应时间不超过 2 秒”属于什么需求？

答案：非功能需求。

题 3： 判断：需求分析只影响开发，不影响测试。

答案：错。测试目标、范围和通过标准都应依据需求规格说明。

3.3 概要设计与详细设计

必须掌握

比较项	概要设计	详细设计
关注点	系统整体结构	模块内部实现
主要问题	系统怎么分	每个模块怎么做
常见任务	模块划分、接口设计、体系结构设计	算法设计、数据结构设计、控制流程设计
常见产物	软件结构图、模块结构图	详细设计说明书、N-S 图、PAD 图、流程图

易错点

错误说法	正确说法
概要设计给出模块内部算法	模块内部算法属于详细设计
详细设计确定系统总体结构	总体结构属于概要设计
需求分析决定模块划分	模块划分属于概要设计

练习题与答案

题 1： 软件结构图通常是哪一阶段的产物?

答案：概要设计。

题 2： N-S 图和 PAD 图常用于哪个阶段?

答案：详细设计。

题 3： 判断：概要设计也称总体设计。

答案：对。

四、软件过程模型

4.1 过程模型总览

模型	核心特点	适用场景
瀑布模型	阶段顺序、线性推进、文档驱动	需求明确稳定、变化少
增量模型	分批开发、分批交付	系统可拆成多个功能增量
演化模型	多次迭代、逐步完善	需求不明确、需要用户反馈
螺旋模型	风险驱动，瀑布 + 演化 + 风险分析	大型、复杂、高风险项目
喷泉模型	面向对象，迭代、无间隙	面向对象开发
敏捷模型	迭代增量，快速反馈，拥抱变化	需求变化快、强调协作

4.2 瀑布模型

必须掌握

典型流程：

```
需求分析 → 概要设计 → 详细设计 → 编码 → 测试 → 运行维护
```

特点：

1. 阶段划分清楚。
2. 每个阶段有明确输入和输出。
3. 前一阶段完成后再进入后一阶段。
4. 适合需求明确、变化少的项目。
5. 不适合需求频繁变化的大型复杂项目。

练习题与答案

题 1： 瀑布模型为什么适合需求稳定项目？

答案：因为瀑布模型按阶段顺序推进，前期需求一旦确定后，后续设计、编码、测试依次展开，需求变化少时返工成本较低。

题 2： 判断：瀑布模型强调各阶段可以自由交叉进行。

答案：错。瀑布模型强调阶段顺序和文档驱动。

4.3 增量模型与演化模型

必须掌握

增量模型：

> 把系统分成若干功能增量，每次开发并交付一部分功能。

演化模型：

> 通过多次迭代和用户反馈，让软件从初始版本逐步演化为最终系统。

区别：

模型	重点
增量模型	分批交付功能
演化模型	根据反馈不断修改完善

练习题与答案

题 1： 一个教务系统先交付登录，再交付选课，再交付缴费，属于什么模型？

答案：增量模型。

题 2： 需求不清楚，需要先做原型给用户试用再逐步修改，适合什么模型？

答案：演化模型或原型化思想。

4.4 螺旋模型

必须掌握

螺旋模型模板：

> 螺旋模型将瀑布模型和演化模型结合起来，并增加了风险分析。它以原型为基础，每进行一轮螺旋式开发，就得到软件的

四个典型活动：

1. 制定计划。
2. 风险分析。
3. 工程实现。
4. 客户评估。

高频考法

问法	答案
螺旋模型增加了什么	风险分析
螺旋模型建立在什么基础上	原型
每旋转一圈得到什么	软件的一个新版本
适合什么项目	大型、复杂、高风险项目

练习题与答案

题 1： 螺旋模型的核心特点是什么？

答案：风险驱动，强调每轮迭代中的风险分析。

题 2： 单选：螺旋模型在瀑布模型和演化模型基础上增加了什么？

A. 模块划分
B. 风险分析
C. 数据库设计
D. 编码规范

答案：B。

4.5 喷泉模型

必须掌握

喷泉模型模板：

> 喷泉模型是面向对象开发模型，体现了面向对象开发中迭代和无间隙的特征。它不严格区分分析、设计和实现阶段，而分

关键词：

> 面向对象、迭代、无间隙、不严格线性

练习题与答案

题 1： 喷泉模型主要适用于哪类开发方法?

答案：面向对象开发。

题 2： 判断：喷泉模型像瀑布模型一样严格按阶段线性推进。

答案：错。喷泉模型强调迭代和无间隙。

4.6 敏捷模型

必须掌握

敏捷开发强调：

1. 迭代增量交付。
2. 快速响应需求变化。
3. 用户持续参与。
4. 团队协作。
5. 可工作的软件比过度文档更重要。

易错点

错误说法	正确说法
敏捷完全不需要文档	敏捷不是不要文档，而是避免过度文档

错误说法	正确说法
敏捷适合需求固定且严格监管项目	敏捷更适合需求变化快、反馈频繁项目
敏捷一次性交付完整系统	敏捷强调迭代增量交付

练习题与答案

题 1： 敏捷开发适合什么类型项目？

答案：需求变化较快、需要频繁用户反馈、可迭代交付的项目。

题 2： 判断：敏捷开发强调拥抱变化。

答案：对。

4.7 RUP

必须掌握

RUP，Rational Unified Process，统一软件开发过程，是迭代式、增量式的软件过程。

RUP 核心概念：

问题	RUP 概念
谁做	角色
做什么	活动/任务
产生什么	制品
怎么组织	工作流

6 个核心工程工作流：

1. 业务建模。
2. 需求。
3. 分析与设计。
4. 实现。
5. 测试。
6. 部署。

3 个核心支持工作流：

1. 项目管理。
2. 配置与变更管理。
3. 环境。

答题模板

```
RUP 的核心工程工作流包括业务建模、需求、分析与设计、实现、测试和部署。业务建模理解业务流程；需求定义功能
```

练习题与答案

题 1： RUP 中系统分析员、设计员、测试员属于什么概念?

答案：角色。

题 2： RUP 中用例模型、设计模型、测试报告属于什么概念?

答案：制品。

题 3： RUP 中负责安装、配置、用户培训和软件交付的是哪个工作流?

答案：部署工作流。

4.8 RAD

必须掌握

RAD，Rapid Application Development，快速应用程序开发。

RAD 的三种建模：

建模	主要活动
业务建模	分析业务流程和信息流
数据建模	识别数据对象、属性和关系
流程建模	描述数据如何被增加、修改、删除、查询和加工

答题模板

> RAD 包括业务建模、数据建模和流程建模。业务建模分析业务职能之间的信息流；数据建模识别数据对象、属性及其关

练习题与答案

题 1： RAD 中识别实体、属性和实体关系属于哪种建模?

答案：数据建模。

题 2： RAD 中分析业务信息流属于哪种建模?

答案：业务建模。

五、面向对象方法与 UML

5.1 类、对象、属性、方法

必须掌握

概念	标准理解
类	对具有相同属性和行为的一组对象的抽象描述
对象	类的具体实例
属性	对象具有的数据特征
方法/操作	对象能够执行的行为

必背句：

> 类是对象的抽象，对象是类的实例。

练习题与答案

题 1： 在面向对象方法中，类是对象的什么？对象是类的什么？

答案：类是对象的抽象，对象是类的实例。

题 2： “学生”和“张三”的关系是什么？

答案：“学生”是类，“张三”是对象。

题 3： 判断：对象只包含数据，不包含操作。

答案：错。对象由属性和操作组成。

5.2 封装、继承、多态、重载、重写

必须掌握

概念	关键词	例子
封装	数据和操作绑定、信息隐藏、接口访问	私有属性通过方法访问
继承	子类复用父类属性和方法	学生继承人
多态	同一消息、不同对象、不同响应	Dog.speak 与 Cat.speak
重载	同名方法、参数不同、编译期决定	print(int), print(String)
重写	子类覆盖父类方法、运行时绑定	子类重新定义 speak

解题技巧

1. 看到“隐藏对象内部实现”，选封装。
2. 看到“子类复用父类”，选继承。
3. 看到“同一消息，不同对象，不同响应”，选多态。
4. 看到“同名方法，参数列表不同”，选重载。
5. 看到“子类重新定义父类方法”，选重写。

练习题与答案

题 1： 同一操作作用于不同对象，产生不同执行结果，这种机制称为什么？

答案：多态。

题 2： 方法名相同、参数列表不同，称为什么？

答案：重载。

题 3： 子类重新定义父类已有方法，称为什么？

答案：重写。

5.3 消息传递与动态绑定

必须掌握

面向对象系统中，对象之间通过消息传递进行协作。

> 发送者发送消息，请求接收对象执行某个操作；发送者只需知道对象接口，不必知道内部实现。

消息传递与传统过程调用区别：

比较项	消息传递	过程调用
所属思想	面向对象	面向过程
调用目标	对象接口	函数/过程
绑定方式	可支持动态绑定	通常较固定
设计价值	支持封装、多态	支持功能分解

答题模板

> 消息传递是面向对象方法中对象之间的通信机制。发送者只需要知道接收对象提供的接口，不必了解其内部实现，因此支

练习题与答案

题 1： 面向对象方法中，对象之间的交互通过什么实现？

答案：消息传递。

题 2： 判断：接收消息的对象必须回答消息发送者。

答案：错。对象收到消息后可以执行相应操作，不一定必须返回回答。

5.4 对象状态与生命周期

必须掌握

对象不是创建后一直处于活动状态，可能经历：

```
创建 → 使用/等待 → 不再引用 → 回收
```

对象状态表示对象在某一时刻的条件或情况，状态会因事件而改变。

练习题与答案

题 1： 判断：程序运行时所有已创建对象都始终处于活动状态。

答案：错。对象可能等待、不可达或被回收。

题 2： 对象状态变化通常由什么触发?

答案：事件或消息。

5.5 面向对象分析三大模型

必须掌握

模型	描述角度	回答问题	常见工具
对象模型	静态结构	系统中有什么	类图、对象图
动态模型	动态行为	系统怎样运行、对象如何变化	状态图、顺序图、活动图
功能模型	功能与数据处理	系统要做什么	用例图、DFD

答题模板

```
对象模型描述系统的静态结构，说明系统中有哪些对象、类及其关系；动态模型描述系统的动态行为，说明对象如何响应
```

练习题与答案

题 1： 对象模型主要描述系统的哪一方面?

答案：静态结构。

题 2： 动态模型常用哪些图表示?

答案：状态图、顺序图、活动图。

题 3： 判断：功能模型主要描述类之间的继承关系。

答案：错。类之间的静态关系主要属于对象模型，功能模型描述功能和数据流。

5.6 4+1 视图模型

必须掌握

视图	描述内容	记忆关键词
逻辑视图	类、对象、功能关系	功能
开发视图	模块、包、代码组织	代码
物理视图	服务器、网络节点、部署结构	部署
进程视图	进程、线程、并发、通信	运行
场景视图/用例视图	用例串联和验证其他视图	用例

记忆口诀：

> 逻辑看功能，开发看代码，物理看部署，进程看运行，场景看用例。

练习题与答案

题 1： 描述服务器、网络节点和部署关系的是哪个视图?

答案：物理视图。

题 2： 描述进程、线程和并发通信的是哪个视图?

答案：进程视图。

5.7 UML 类间关系

必须掌握

关系	含义	例子
依赖	一个类临时使用另一个类	A 的方法参数使用 B
关联	两个类存在结构性联系	学生选课
聚合	弱整体/部分，部分可独立存在	班级和学生
组合	强整体/部分，生命周期依赖整体	订单和订单明细
泛化	继承关系	学生是人
实现	类实现接口	类实现 Runnable

易错点

整体/部分关系通常用聚合或组合，不用依赖。
组合比聚合更强，部分对象通常不能脱离整体单独存在。

练习题与答案

题 1： 班级与学生通常是什么关系？

答案：聚合关系。

题 2： 订单与订单明细通常是什么关系？

答案：组合关系。

题 3： 子类和父类之间是什么 UML 关系？

答案：泛化关系。

5.8 UML 状态图

必须掌握

状态图用于描述对象或系统在事件驱动下的状态变化。

核心元素：

1. 初始状态。
2. 状态。
3. 转移。
4. 事件。
5. 条件/守卫条件。
6. 动作。
7. 终止状态。

常见表示：

```
状态A --事件[条件]/动作--> 状态B
```

解题步骤

1. 找状态：显示、等待、输入、验证、确认、处理、成功、失败。
2. 找事件：点击、提交、输入、取消、继续、结束。
3. 找条件：正确/错误、有效/无效、成功/失败。
4. 找异常路径：错误返回、取消返回、失败重试。
5. 补全状态名：根据箭头前后动作和条件命名。

常见满分路径

以成绩查询系统为例：

显示首页 → 要求学号 → 校验学号
校验学号错误 → 显示错误信息 → 显示首页
校验学号正确 → 要求课程名 → 校验课程名
课程名错误 → 显示错误信息 → 显示首页
课程名正确 → 查询选课文件
查询失败 → 显示查询失败信息 → 显示首页
查询成功 → 显示成绩 → 询问用户操作
继续查询 → 要求学号
结束查询 → 显示首页/终止

练习题与答案

题 1： 用户输入学号后，系统检查学号是否存在，这个状态可命名为什么？

答案：校验学号。

题 2： 用户输入错误课程名后，系统显示错误并返回首页，至少涉及哪些状态？

答案：校验课程名、显示错误信息、显示首页。

题 3： 判断：状态图只描述静态类结构。

答案：错。状态图描述动态状态变化。

5.9 UML 顺序图

必须掌握

顺序图用于描述对象之间消息传递的时间顺序。

基本元素：

元素	含义
参与者	系统外部角色
对象	参与交互的对象
生命线	对象在交互过程中的存在时间
消息	对象之间发送的请求

元素	含义
返回消息	请求返回结果
激活条	对象正在执行操作的时间段

时间方向：

> 从上到下。

顺序图文字答题模板

> 参与对象包括……。首先……向……发送……消息；然后……查询/处理……；若条件成立，则……；否则……。消息按照从上到下的

练习题与答案

题 1： 顺序图主要描述什么？

答案：对象之间按时间顺序发生的消息交互过程。

题 2： 顺序图中时间方向通常怎样？

答案：从上到下。

六、软件设计：内聚、耦合、结构化程序

6.1 高内聚低耦合

必须掌握

内聚表示模块内部各部分联系的紧密程度，内聚越高越好。

耦合表示模块之间相互依赖的程度，耦合越低越好。

> 优秀模块设计追求：高内聚、低耦合。

6.2 内聚类型

必须掌握

内聚	含义	例子
偶然内聚	模块内部功能毫无关系	把无关工具函数放一起
逻辑内聚	逻辑上相似，通过控制参数选择功能	opType 决定增删改查
时间内聚	因为同一时间执行而放一起	初始化时打开文件、读配置
过程内聚	按固定步骤执行，但数据联系不强	先验证再打印再记录
通信内聚	多个操作使用同一输入或输出数据	都处理同一客户资料
顺序内聚	前一部分输出是后一部分输入	读取订单→计算金额→生成发票
功能内聚	只完成一个明确功能	计算订单总价

高频考法

通过控制参数决定执行哪个功能：逻辑内聚。
前一步输出作为后一步输入：顺序内聚。
所有操作围绕一个明确功能：功能内聚。
多个操作使用同一数据：通信内聚。

练习题与答案

题 1： 某模块通过 `opType` 参数决定执行添加、删除、修改、查询，属于哪种内聚？

答案：逻辑内聚。

题 2： “读取订单信息 → 计算订单金额 → 生成发票”属于哪种内聚？

答案：顺序内聚。

题 3： 最高内聚是哪种？

答案：功能内聚。

6.3 耦合类型

必须掌握

耦合	含义
非直接耦合	两模块无直接关系
数据耦合	通过简单参数传递数据
标记耦合	传递复合数据结构
控制耦合	传递控制参数影响对方逻辑
外部耦合	共同依赖外部格式、设备或协议
公共耦合	多模块访问同一全局数据
内容耦合	一个模块直接访问或修改另一个模块内部

高频考法

只传递必要简单数据：数据耦合，较好。
传递标志变量控制对方分支：控制耦合。
多个模块共用全局变量：公共耦合。
一个模块直接访问另一个模块内部数据：内容耦合，最差。

练习题与答案

题 1： 模块 A 通过参数把 `studentId` 传给模块 B，属于什么耦合？

答案：数据耦合。

题 2： 模块 A 传入 `flag` 控制模块 B 执行新增还是删除，属于什么耦合？

答案：控制耦合。

题 3： 判断：模块之间耦合越高越好。

答案：错。应追求低耦合。

6.4 设计模式

必须掌握

设计模式是：

> 针对反复出现的软件设计问题总结出的可复用解决方案。

核心价值：

1. 复用成熟设计经验。
2. 提高可维护性。
3. 提高可扩展性。
4. 降低设计复杂度。
5. 与具体语言无关。

易错点

> 设计模式不是为了直接提高程序运行速度。
> 设计模式不是只适用于 `Java`。
> 设计模式不是保证程序员一定更快写完代码。

练习题与答案

题 1： 设计模式的核心作用是什么？

答案：复用成熟设计经验，解决常见设计问题。

题 2： 单例模式、工厂方法模式、观察者模式都属于什么？

答案：设计模式。

6.5 结构化程序设计三种基本控制结构

必须掌握

结构化程序设计要求复杂程序由三种基本控制结构组合实现：

1. 顺序结构。

2. 选择结构。
3. 循环结构。

特点：

```
单入口、单出口、逻辑清晰、避免随意 goto。
```

类 C 表示：

```
// 顺序结构
A();
B();

// 选择结构
if (condition) {
    A();
} else {
    B();
}

// 循环结构
while (condition) {
    A();
}
```

练习题与答案

题 1： 结构化程序设计的三种基本控制结构是什么？

答案：顺序结构、选择结构、循环结构。

题 2： 为什么强调单入口、单出口？

答案：提高程序可读性、可测试性和可维护性，降低控制流复杂度。

6.6 N-S 图与 PAD 图

必须掌握

N-S 图和 PAD 图都常用于详细设计阶段，表示程序控制结构。

图	特点
N-S 图	盒图，强制结构化，适合表示顺序、选择、循环
PAD 图	问题分析图，层次清晰，便于转换为程序

三种基本结构：

```
顺序：A 后执行 B。
选择：if 条件 then A else B。
循环：while 条件 do A。
```

解题技巧

1. 先把题干伪代码缩进清楚。
2. 再识别顺序、选择、循环。
3. 作图时保持单入口单出口。
4. 不要把循环体和循环条件放反。

练习题与答案

题 1： N-S 图和 PAD 图常用于哪个阶段?

答案：详细设计。

题 2： 判断：N-S 图可以表示顺序、选择、循环三种结构。

答案：对。

七、软件测试

7.1 软件测试目的与原则

必须掌握

软件测试的目的是：

发现程序中的错误，而不是证明程序没有错误。

经典判断：

好的测试用例是能够发现尚未发现错误的测试用例。
成功的测试是发现了错误的测试。
测试只能证明缺陷存在，不能证明缺陷不存在。

答题模板

软件测试的目的是发现程序中的错误，而不是证明程序没有错误。测试通过执行程序，检查软件是否满足需求，帮助提高

练习题与答案

题 1： 软件测试的主要目的是什么？

答案：发现程序中的错误。

题 2： 判断：软件测试可以证明程序没有错误。

答案：错。测试只能证明错误存在，不能证明错误不存在。

7.2 黑盒测试与白盒测试

必须掌握

黑盒测试：

1. 不关心程序内部结构。
2. 根据需求规格说明设计测试。
3. 关注输入、输出、功能是否正确。
4. 常用方法：等价类划分、边界值分析、因果图、判定表。

白盒测试：

1. 关注程序内部逻辑。
2. 测试语句、分支、条件、路径。
3. 常用方法：语句覆盖、判定覆盖、条件覆盖、路径覆盖、基本路径测试。

高频判断

边界值分析属于黑盒测试。
基本路径测试属于白盒测试。
黑盒测试主要考虑功能。
白盒测试主要考虑程序内部逻辑。

练习题与答案

题 1： 输入年龄范围 18-60，测试 17、18、60、61，属于哪种方法？

答案：边界值分析，属于黑盒测试。

题 2： 根据代码 `if-else` 分支设计用例，属于什么测试？

答案：白盒测试。

7.3 测试阶段分类

必须掌握

阶段	测试对象	主要目标
单元测试	单个模块、函数、类	发现模块内部逻辑错误
集成测试	多个模块组合	发现接口、调用、协作错误
系统测试	完整软件系统	验证是否满足需求规格说明
验收测试	即将交付的软件	用户确认是否可交付

易错点

单元测试主要发现模块内部错误。
集成测试主要发现模块接口之间的错误。
验收测试通常由用户或客户参与。

练习题与答案

题 1： 发现模块之间接口错误的主要测试阶段是？

答案：集成测试。

题 2： 银行系统开发完成后，由客户根据合同确认是否上线，属于什么测试？

答案：验收测试。

7.4 覆盖准则与最少测试用例

必须掌握

覆盖准则	要求	强度
语句覆盖	每条可执行语句至少执行一次	最弱
判定覆盖/分支覆盖	每个判断真假分支都至少一次	较强
条件覆盖	每个条件真假都至少一次	较强
路径覆盖	所有可能路径至少一次	最强但代价高

解题技巧

1. 语句覆盖找“怎样让所有语句都执行”，用例通常最少。
2. 判定覆盖每个 `if` 的真/假都要出现。
3. 条件覆盖关注复合条件中每个小条件的真/假。
4. 路径覆盖路径数量可能很多，考试常考基本路径覆盖。

练习题与答案

题 1： 下面程序满足语句覆盖最少几个用例？

```
if (x > 0) {
    y = 1;
}
print(y);
```

答案：1 个，例如 `x = 1` 。

题 2： 下面程序满足判定覆盖最少几个用例？

```
if (x > 0) {
    y = 1;
} else {
    y = -1;
}
```

答案：2 个，例如 `x = 1` 覆盖真分支， `x = 0` 覆盖假分支。

题 3： 为下面程序设计一个满足语句覆盖的测试用例。

```
min = a;
if (b < min) min = b;
if (c < min) min = c;
print(min);
```

答案： `a = 5, b = 3, c = 1` ，预期输出 `1` 。

7.5 McCabe 环路复杂度与基本路径覆盖

必须掌握

McCabe 环路复杂度用于度量程序控制流复杂度。

常用公式：

```
V(G) = 判定节点数 + 1
V(G) = E - N + 2
```

如果有多个连通区域：

```
V(G) = E - N + 2P
```

判定节点包括：

```
if、while、for、switch 分支、条件判断、状态图中的分支判断。
```

基本路径覆盖答题模板

```
先计算 V(G)，得到至少需要的独立路径条数。
再列出每条独立路径，确保每个判断的不同分支都被覆盖。
```

以成绩查询为例，有 4 个判断：

```
学号是否正确
课程名是否正确
查询是否成功
用户继续还是结束
```

所以：

```
V(G) = 4 + 1 = 5
```

5 条基本路径：

```
1. 学号错误路径：
首页 → 输入学号 → 校验学号 → 显示学号错误 → 首页

2. 课程名错误路径：
首页 → 输入学号 → 学号正确 → 输入课程名 → 校验课程名错误 → 首页

3. 查询失败路径：
首页 → 输入学号 → 学号正确 → 输入课程名 → 课程名正确 → 查询选课文件 → 查询失败 → 首页

4. 查询成功后结束路径：
首页 → 输入学号 → 学号正确 → 输入课程名 → 课程名正确 → 查询成功 → 显示成绩 → 结束查询 → 首页

5. 查询成功后继续路径：
首页 → 输入学号 → 学号正确 → 输入课程名 → 课程名正确 → 查询成功 → 显示成绩 → 继续查询 → 输入学号
```

练习题与答案

题 1： 某程序有 3 个 `if` 判断，没有循环，McCabe 环路复杂度是多少？

答案： `3 + 1 = 4` 。

题 2： 某流程有用户名、密码、验证码三个判断，至少需要几条基本路径？

答案：4 条。

题 3： 基本路径覆盖的目的是什么？

答案：设计一组测试用例，使程序中的每条独立路径至少执行一次。

7.6 测试用例设计

必须掌握

测试用例不应只包含合理输入，还应包括：

1. 合理输入。
2. 不合理输入。
3. 边界输入。
4. 异常输入。

高频判断

只测试正常输入是不充分的。
边界值通常比中间值更容易发现错误。
测试阶段主要执行测试计划，不是才开始制定测试计划。

练习题与答案

题 1： 年龄范围 18-60，合理的边界测试数据有哪些？

答案：17、18、60、61。

题 2： 判断：测试用例只需要覆盖用户正常输入。

答案：错。还要覆盖非法、边界、异常输入。

八、结构化分析与 DFD 建模

8.1 结构化分析方法

必须掌握

结构化分析方法是一种面向数据流的软件分析方法，通常采用：

> 自顶向下、逐层分解、逐步求精。

特点：

1. 先分析系统整体。
2. 再逐层分解内部功能。
3. 用 DFD 描述数据流动和加工。
4. 适合数据处理型系统。

练习题与答案

题 1： 结构化分析方法通常采用什么思想?

答案：自顶向下、逐层分解、逐步求精。

题 2： 判断：结构化分析方法通常从最底层模块开始分析。

答案：错。应从整体开始，自顶向下分解。

8.2 DFD 四元素

必须掌握

DFD，Data Flow Diagram，数据流图，用于描述系统中数据如何流动、加工和存储。

四个基本元素：

1. 外部实体。
2. 加工。
3. 数据流。

4. 数据存储。

易错点

易错点	正确做法
外部实体直接访问数据存储	必须经过加工
数据流不命名	每条数据流都应有名称
把数据库画成外部实体	数据库通常是数据存储
顶层 DFD 画很多内部模块	顶层只画一个系统加工

练习题与答案

题 1： DFD 包含哪四种基本元素？

答案：外部实体、加工、数据流、数据存储。

题 2： 在 DFD 中，“学生”“课程文件”“提交订单”分别属于什么？

答案：学生是外部实体，课程文件是数据存储，提交订单是数据流。

8.3 顶层 DFD、0 层 DFD、1 层 DFD

必须掌握

层次	含义
顶层 DFD/环境图	把整个系统看成一个加工，只显示系统和外部实体之间的数据流
0 层 DFD	将顶层总加工分解为若干主要加工
1 层 DFD	对 0 层中的某个加工继续细化
底层 DFD	不能或不需要继续分解的详细图

顶层 DFD 画法

中间：一个系统加工。
周围：外部实体。
箭头：外部实体与系统之间的数据流。
不画：内部数据存储、内部处理细节。

0 层 DFD 画法

将系统分成几个主要加工。
画出必要数据存储。
保持与顶层 DFD 的输入输出平衡。

1 层 DFD 画法

选择 0 层中的某个复杂加工继续分解。
保留该加工在父图中的输入输出。
细化内部子加工和数据流。

练习题与答案

题 1： 顶层 DFD 又叫什么？

答案：系统环境图。

题 2： 顶层 DFD 中整个系统通常表示为什么？

答案：一个加工。

题 3： 0 层 DFD 和顶层 DFD 的关系是什么？

答案：0 层 DFD 是对顶层 DFD 中总加工的进一步分解。

8.4 DFD 数据平衡原则

必须掌握

数据平衡原则：

父图中的输入输出数据流，在子图中必须能够对应，不能凭空增加或减少。

解题技巧

1. 先列父图输入输出。
2. 再检查子图是否都有来源和去向。
3. 子图可以细化内部数据流，但不能改变对外输入输出。

练习题与答案

题 1： 什么是 DFD 数据平衡原则?

答案：父图和子图的输入输出数据流应保持一致，不能无故增加或遗漏。

题 2： 判断：0 层 DFD 可以随意增加顶层 DFD 中没有出现的外部输入。

答案：错。必须保持数据平衡。

8.5 数据字典

必须掌握

数据字典用于说明 DFD 中数据流、数据存储、数据项的组成。

简单理解：

```
DFD 画出数据怎么流，数据字典说明数据里面有什么。
```

常用符号：

符号	含义	示例
=	定义为	罚款单 = 借阅人编号 + 金额
+	由……组成	图书卡片 = 书名 + 作者
`[	]`	选择其一
{}	重复	借阅记录 = {借书记录}
()	可选	学生信息 = 学号 + 姓名 +（电话）

答题模板

```
数据流名称 = 数据项1 + 数据项2 + 数据项3
```

例如：

```
罚款单 = 借阅人图书证号 + 图书流水号 + 罚款金额
月末库存量 = 分类目录号 + 书名 + 库存量
```

练习题与答案

题 1： 数据字典的作用是什么？

答案：定义 DFD 中数据流、数据存储和数据项的组成，使数据含义明确统一。

题 2： 写出“图书卡片”的数据字典定义。

答案：

```
图书卡片 = 分类目录号 + 流水号 + 书名 + 作者 + 价格 + 购书日期
```

8.6 DFD 建模通用步骤

必须掌握

遇到 DFD 建模题，按四步：

1. 找外部实体：谁向系统提供数据，谁接收系统输出。
2. 找数据存储：题干中的文件、数据库、记录表。
3. 找加工：动词短语，如报名处理、注销处理、付款处理、借书处理。
4. 找数据流：输入输出信息、单据、通知、查询请求、结果。

满分检查清单

```
外部实体是否在系统边界外?
每个加工是否有输入和输出?
数据流是否命名?
外部实体是否没有直接连数据存储?
父图和子图是否数据平衡?
顶层 DFD 是否只画一个系统加工?
```

练习题与答案

题 1： 某教务系统：学生提交选课申请，系统检查课程是否已满；未满则登记学生文件和课程文件，输出成功通知；已满则输出失败通知。列出外部实体、数据存储、加工、数据流。

答案：

```
外部实体：学生
数据存储：学生文件、课程文件
加工：选课处理、课程容量检查、登记选课信息
数据流：选课申请、成功通知、失败通知、课程容量信息、学生选课记录
```

题 2： 培训中心报名处理涉及哪些数据存储?

答案：学生文件、课程文件、账目文件。

题 3： 图书管理系统中“借阅人”通常属于什么元素?

答案：外部实体。

九、过程改进、配置管理、风险与维护

9.1 CMM 五级模型

必须掌握

等级	名称	核心特征
1 级	初始级	过程混乱，依赖个人能力
2 级	可重复级	建立基本项目管理过程，能重复以往成功经验
3 级	已定义级	组织级过程标准化、文档化
4 级	已管理级	使用定量方法管理过程和质量
5 级	优化级	持续改进过程，预防缺陷

第 2 级关键过程域常考：

```
需求管理
软件项目规划
软件项目跟踪和监督
软件转包管理
软件质量保证
软件配置管理
```

答题模板

```
CMM 五级模型包括初始级、可重复级、已定义级、已管理级和优化级。以可重复级为例，其关键过程域包括需求管理、
```

练习题与答案

题 1： CMM 五级模型分别是什么？

答案：初始级、可重复级、已定义级、已管理级、优化级。

题 2： CMM 第 2 级叫什么？列出三个关键过程域。

答案：可重复级。关键过程域包括需求管理、软件项目规划、软件配置管理等。

9.2 CMMI

必须掌握

CMMI，Capability Maturity Model Integration，能力成熟度模型集成，用于软件过程改进。

两种表示方式：

表示方式	关注点
阶段式表示	组织整体成熟度等级
连续式表示	单个过程域能力等级

高频判断：

```
CMMI 既支持阶段式过程改进，也支持连续式过程改进。
```

练习题与答案

题 1： CMMI 是否只支持阶段式改进?

答案：不是。CMMI 同时支持阶段式和连续式改进。

题 2： 阶段式 CMMI 更关注什么?

答案：组织整体成熟度等级。

9.3 软件配置管理与质量控制

必须掌握

软件配置管理 SCM 主要活动：

1. 配置项标识。
2. 版本控制。
3. 变更控制。
4. 配置状态报告。
5. 配置审计。
6. 发布管理。

软件质量控制 QC 主要活动：

1. 测试。
2. 评审。
3. 缺陷发现。
4. 质量度量。
5. 验证产品是否符合质量要求。

高频判断

> 质量控制不是软件配置管理活动。
> 配置管理关注版本、变更、配置项和审计。
> 质量控制关注测试、评审和缺陷发现。

练习题与答案

题 1： Git 版本管理、分支管理、变更审批属于什么活动?

答案：软件配置管理。

题 2： 通过测试发现缺陷，检查软件是否满足质量要求，属于什么活动?

答案：软件质量控制。

9.4 软件风险管理

必须掌握

风险管理一般包括：

1. 风险识别：找出可能发生的问题。
2. 风险预测/评估：分析发生概率和影响程度。
3. 风险应对：规避、转移、缓解、接受。
4. 风险监控：持续跟踪风险状态。

高频判断：

> 风险预测主要看风险发生的可能性和影响程度。

练习题与答案

题 1： “需求频繁变更可能导致项目延期”属于什么风险？

答案：需求风险或项目风险。

题 2： 风险预测主要分析哪两个因素？

答案：风险发生概率和风险影响程度。

9.5 软件维护

必须掌握

维护类型	含义	例子
纠错性维护	修复交付后发现的错误	修复金额计算 bug
适应性维护	适应外部环境变化	适配新版操作系统
完善性维护	增强功能、改善性能、提升体验	增加批量导入
预防性维护	提高未来可维护性和可靠性	重构代码、补充测试

高频判断

软件维护包括纠错性维护，不只是功能完善。

练习题与答案

题 1： 上线后发现订单金额计算错误并修复，属于什么维护？

答案：纠错性维护。

题 2： 为适配新版数据库修改软件，属于什么维护？

答案：适应性维护。

题 3： 重构代码以降低未来维护成本，属于什么维护？

答案：预防性维护。

十、补充高频考点

10.1 开源软件

必须掌握

开源软件是：

```
源代码可以被公众获取、查看、使用、修改和分发，并且通常遵守某种开源许可证的软件。
```

常见许可证：

```
GPL、MIT、Apache License、BSD。
```

典型例子：

```
Linux、MySQL、PostgreSQL、Python、Kubernetes。
```

易错点：

```
开源不一定等于免费。
免费软件不一定开源。
开源软件也要遵守许可证。
```

练习题与答案

题 1： 什么是开源软件？举一个例子。

答案：源代码可以被公众获取、查看、修改和分发的软件。例如 Linux。

题 2： 免费软件一定是开源软件吗？

答案：不一定。免费只表示不收费，不代表源代码开放。

10.2 Web 应用软件

必须掌握

Web 应用软件是：

```
基于浏览器/服务器 B/S 架构，通过 HTTP/HTTPS 协议运行，用户使用浏览器访问，功能和数据主要部署在服务器端
```

基本组成：

```
浏览器端：HTML、CSS、JavaScript
服务器端：Java、Python、Go、Node.js、PHP 等
数据库：MySQL、PostgreSQL、Oracle、MongoDB 等
网络协议：HTTP/HTTPS
```

特点：

```
无需安装专用客户端、跨平台、集中部署、易维护、便于升级、依赖网络。
```

练习题与答案

题 1： 什么是 Web 应用软件?

答案：Web 应用软件是基于 B/S 架构，通过 HTTP/HTTPS 协议运行，用户使用浏览器即可访问的软件系统。

题 2： 列举两个 Web 应用软件例子。

答案：淘宝、京东、新闻网站、网上银行、在线教育平台等。

10.3 编程风格

必须掌握

良好编程风格包括：

1. 命名清晰。
2. 缩进统一。

3. 结构清楚。
4. 注释适当。
5. 避免过长函数。
6. 避免不必要的全局变量。
7. 程序可读、可维护。

高频判断：

```
良好编程风格有助于软件维护。
注释不是越多越好，应说明关键逻辑和意图。
```

练习题与答案

题 1： 良好编程风格的目的是什么？

答案：提高程序可读性、可理解性和可维护性。

题 2： 判断：注释越多，程序质量一定越高。

答案：错。注释应准确、必要，过多无效注释反而降低可读性。

十一、按题型解题技巧

11.1 选择题

选择题主要考概念边界。优先背这些对应关系：

```
黑盒测试：按需求测功能。
白盒测试：按代码测逻辑。
单元测试：模块内部。
集成测试：模块接口。
概要设计：总体结构、模块关系。
详细设计：模块算法、内部流程。
多态：同一消息，不同对象，不同响应。
重载：同名方法，参数不同。
逻辑内聚：控制参数选择功能。
顶层 DFD：系统与外部实体的输入输出。
```

11.2 判断题

看到绝对化说法要警惕：

```
一定、始终、全部、只包括、不包括、必须、都、永远。
```

高频判断结论：

```
CMMI 支持阶段式和连续式：对。
对象创建后始终活动：错。
维护不包括纠错：错。
单元测试主要发现接口错误：错。
耦合越高越好：错。
外部实体可以直接连数据存储：错。
测试能证明程序无错：错。
```

11.3 简答题

简答题建议用：

```
概念 + 特点 + 示例
```

或：

```
名称 + 主要活动 + 说明
```

例：回答 RAD：

```
RAD 包括业务建模、数据建模和流程建模。业务建模分析业务信息流；数据建模识别数据对象、属性和关系；流程建模
```

11.4 建模题

建模题先列清单再画图：

DFD：外部实体、加工、数据流、数据存储。
状态图：状态、事件、条件、异常路径、返回路径。
顺序图：参与者、对象、消息、返回消息、条件分支。
McCabe：判断节点数 + 1，再列基本路径。

十二、综合练习题

12.1 选择题

题 1： 软件需求分析阶段主要解决的问题是：

A. 软件怎样实现
B. 软件要做什么
C. 软件如何测试
D. 软件如何部署

答案：B。

题 2： 概要设计主要确定：

A. 模块内部算法
B. 源程序代码
C. 系统总体结构和模块关系
D. 测试输入数据

答案：C。

题 3： 黑盒测试主要依据：

A. 程序流程图
B. 源程序代码
C. 需求规格说明书
D. 机器指令

答案：C。

题 4： 螺旋模型的核心特点是：

A. 严格顺序执行
B. 风险分析
C. 无阶段划分
D. 只适合小项目

答案：B。

题 5： 喷泉模型主要适用于：

A. 面向对象开发
B. 面向过程开发
C. 数据库设计
D. 编译器优化

答案：A。

题 6： 在面向对象中，同一消息作用于不同对象产生不同响应称为：

A. 封装
B. 继承
C. 多态
D. 聚合

答案：C。

题 7： 顶层 DFD 中，整个系统通常表示为：

A. 一个加工
B. 一个数据存储
C. 一个外部实体
D. 一个数据流

答案：A。

题 8： 模块通过简单参数传递数据，属于：

A. 数据耦合
B. 控制耦合
C. 公共耦合
D. 内容耦合

答案：A。

12.2 判断题

题 1： 测试的目的是证明程序没有错误。

答案：错。测试的目的是发现错误。

题 2： CMMI 同时支持阶段式和连续式过程改进。

答案：对。

题 3： 类是对象的具体实例。

答案：错。对象是类的具体实例。

题 4： 0 层 DFD 是对顶层 DFD 的进一步分解。

答案：对。

题 5： 外部实体可以直接访问数据存储。

答案：错。必须经过加工。

题 6： 单元测试主要发现模块接口之间的错误。

答案：错。模块接口错误主要由集成测试发现。

题 7： 功能内聚是内聚程度最高的一类。

答案：对。

题 8： 软件维护包括纠错性维护。

答案：对。

12.3 简答题

题 1： 简述黑盒测试和白盒测试的区别。

参考答案：

黑盒测试不关心程序内部结构，依据需求规格说明书设计测试用例，重点检查输入输出和功能是否正确，常用方法有等价

题 2： 简述对象模型、动态模型和功能模型之间的关系。

参考答案：

对象模型描述系统的静态结构，说明系统中有哪些对象、类及其关系；动态模型描述系统的动态行为，说明对象如何响应

题 3： 简述 RUP 的 6 个核心工程工作流。

参考答案：

RUP 的 6 个核心工程工作流包括业务建模、需求、分析与设计、实现、测试和部署。业务建模理解业务流程；需求定义

题 4： 简述 DFD 及其四个基本元素。

参考答案：

DFD 是数据流图，用于描述系统中数据如何流动、加工和存储。它包括四个基本元素：外部实体、加工、数据流和数据

题 5： 简述软件维护的四种类型。

参考答案：

软件维护包括纠错性维护、适应性维护、完善性维护和预防性维护。纠错性维护修复交付后发现的错误；适应性维护使软

12.4 建模题

题 1：DFD 建模

某学校教务系统需要完成以下功能：

学生提交选课申请后，系统检查课程是否已满。
如果课程未满，则在学生文件和课程文件中登记选课信息，并生成选课成功通知。
如果课程已满，则生成选课失败通知。
学生也可以提交退课申请，系统需要修改学生文件和课程文件，并生成退课确认单。

要求：指出外部实体、加工、数据流、数据存储，并给出顶层 DFD 和 0 层 DFD 的文字版。

参考答案：

外部实体：
学生

数据存储：
学生文件、课程文件

主要加工：
选课处理、退课处理、课程容量检查、登记选课信息、修改退课信息

主要数据流：
选课申请、退课申请、选课成功通知、选课失败通知、退课确认单、课程容量信息、学生选课记录

顶层 DFD 文字版：
学生 → 教务系统：选课申请、退课申请
教务系统 → 学生：选课成功通知、选课失败通知、退课确认单

0 层 DFD 文字版：
1. 选课处理接收选课申请，读取课程文件检查课程容量。
2. 若课程未满，则修改学生文件和课程文件，输出选课成功通知。
3. 若课程已满，则输出选课失败通知。
4. 退课处理接收退课申请，修改学生文件和课程文件，输出退课确认单。

题 2：状态图建模

某登录系统流程：

首页显示登录界面。
用户输入用户名和密码。
系统验证用户名和密码。
若验证成功，进入系统主页。
若验证失败，显示登录失败信息，返回登录界面。

要求：列出主要状态和转移。

参考答案：

```
主要状态：
显示登录界面
输入用户名和密码
验证登录信息
显示登录失败信息
进入系统主页

主要转移：
显示登录界面 --输入用户名和密码--> 验证登录信息
验证登录信息 --验证成功--> 进入系统主页
验证登录信息 --验证失败--> 显示登录失败信息
显示登录失败信息 --> 显示登录界面
```

题 3：基本路径覆盖

某流程有如下判断：

```
用户名是否正确
密码是否正确
验证码是否正确
```

要求：计算 McCabe 环路复杂度，并说明至少需要几条基本路径。

参考答案：

```
判定节点数为 3，因此 V(G) = 3 + 1 = 4。
至少需要 4 条基本路径，分别覆盖用户名错误、密码错误、验证码错误、全部正确等独立路径。
```

十三、最后背诵清单

考前至少背熟下面内容：

1. 需求分析：定义需求并建立系统逻辑模型。
2. 概要设计：确定系统总体结构和模块关系。
3. 详细设计：设计模块算法、数据结构和内部流程。
4. 测试计划：通常在需求分析阶段制定。
5. 瀑布模型：需求稳定时适合，需求变化大时不适合。
6. 螺旋模型：瀑布 + 演化 + 风险分析。
7. 喷泉模型：面向对象，迭代，无间隙。
8. 敏捷模型：迭代增量，快速反馈，拥抱变化。
9. RUP：角色、活动、制品、工作流；6 个工程工作流。
10. RAD：业务建模、数据建模、流程建模。
11. 类是对象的抽象，对象是类的实例。
12. 多态：同一消息，不同对象，不同响应。
13. 重载：同名方法，参数不同。
14. 消息传递：对象之间的交互机制。
15. 内聚排序：偶然 < 逻辑 < 时间 < 过程 < 通信 < 顺序 < 功能。
16. 好设计：高内聚、低耦合。
17. 黑盒测试：依据需求，关注功能。
18. 白盒测试：依据程序内部逻辑，关注语句、分支、条件、路径。
19. 单元测试测模块内部，集成测试测模块接口。
20. 测试目的：发现错误，不是证明无错。
21. McCabe：判定节点数 + 1。
22. DFD 四元素：外部实体、加工、数据流、数据存储。
23. 顶层 DFD：系统环境图，只描述系统与外部实体的数据流。
24. 数据字典：数据流 = 数据项 + 数据项。
25. 状态图：状态、事件、转移、条件、动作。
26. 顺序图：按时间顺序描述对象消息交互。
27. CMM 五级：初始级、可重复级、已定义级、已管理级、优化级。
28. CMMI：同时支持阶段式和连续式过程改进。
29. 配置管理：版本、变更、配置项、审计。
30. 风险预测：概率 + 影响。
31. 软件维护：纠错性、适应性、完善性、预防性。
32. 开源软件：源代码开放，但要遵守许可证。
33. Web 应用：B/S 架构，HTTP/HTTPS，浏览器访问，服务器集中部署。

最终复习策略：

第一轮：背速查表和最后背诵清单。
第二轮：重点练 DFD、状态图、McCabe、覆盖准则。
第三轮：用综合练习题检查概念边界，尤其区分需求/设计、黑盒/白盒、单元/集成、内聚/耦合、多态/重载。